



EVEREST ENGINEERING COLLEGE

Pokhara University
Sanepa – 2, Lalitpur

MINI PROJECT REPORT

Customer Support Ticket Router System

A Queue Implementation Using Singly Linked List in C++

Course: Data Structures and Algorithms

Program: Bachelor of Engineering in Computer Engineering

Semester: 2nd Semester | **Language:** C++

Academic Year: 2082 B.S.

SUBMITTED BY

Student Name: AMAN KUMAR SINGH

Roll No.: SEVEN

Exam Roll No.: 25070129

Group / Section: B / B1

SUBMITTED TO

Subject Teacher: PARIS DHUNGANA

Designation:

Date of Submission: __JUNE 23, 2026__

Table of Contents

1. Introduction.....	3
2. Objectives.....	3
3. Theory of Queues.....	4
4. Algorithm: The Ticket Routing Process.....	5
5. Program Logic Flowchart.....	6
6. Complete Source Code.....	7
7. Sample Output.....	11
8. Conclusion.....	12

1. Introduction

Customer support systems are widely used to manage and resolve customer issues in an organized manner. When multiple customers submit requests, the system must ensure that each request is handled fairly and efficiently. One of the most suitable data structures for this purpose is the Queue, which follows the First-In-First-Out (FIFO) principle. According to this principle, the first ticket entered into the system is the first ticket to be resolved.

This project, titled Customer Support Ticket Router System, is a menu-driven C++ application that simulates the management of customer support tickets. The system allows users to add new tickets, view pending tickets, check the next ticket to be processed, resolve tickets in FIFO order, and display the total number of pending tickets.

The queue is implemented using a Singly Linked List, which provides dynamic memory allocation and eliminates the size limitations associated with static arrays. Each ticket is stored as a node in the linked list, and the queue is managed using front and rear pointers.

The main objective of this project is to demonstrate the practical application of the Queue data structure while providing a simple and efficient solution for ticket management.

2. Objectives

The main objectives of the Customer Support Ticket Router System are:

- To understand and implement the Queue data structure using a Singly Linked List.
- To demonstrate the First-In-First-Out (FIFO) principle in a real-world application.
- To develop a menu-driven C++ program for managing customer support tickets.
- To perform basic queue operations such as enqueue (add ticket), dequeue (resolve ticket), peek (view next ticket), display, and count pending tickets.
- To gain practical experience with dynamic memory allocation and pointer manipulation in linked lists.
- To apply data structure concepts to solve a realistic customer service management problem.

3. Theory of Queue (FIFO)

A Queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. This means that the element inserted first is removed first, similar to a line of people waiting for a service. New elements are added at the rear of the queue, while existing elements are removed from the front.

Queues are widely used in operating systems, task scheduling, printer management, network packet handling, and customer support systems. In a customer support environment, requests arrive continuously and must be processed in the order they were received. Therefore, the queue data structure provides a fair and efficient method for ticket management.

The basic operations of a queue are:

- **Enqueue:** Adds a new element at the rear of the queue.
- **Dequeue:** Removes an element from the front of the queue.
- **Peek:** Displays the front element without removing it.
- **isEmpty:** Checks whether the queue contains any elements.
- **Display:** Shows all elements currently stored in the queue.

In this project, the queue is implemented using a Singly Linked List. Each ticket is stored inside a node, and nodes are connected through pointers. The queue is managed using two pointers:

- **Front Pointer:** Points to the first ticket waiting to be resolved.
- **Rear Pointer:** Points to the most recently added ticket.

Using a linked list allows the queue to grow dynamically according to the number of tickets, making the implementation memory-efficient and flexible.

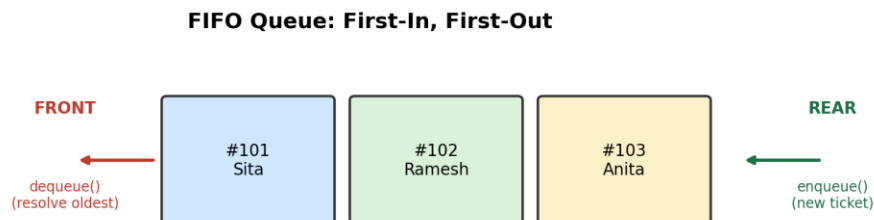


Figure 1. The FIFO queue principle: new tickets enter at the REAR; the support agent always resolves the ticket currently at the FRONT.

4. Data Structures Used

The Customer Support Ticket Router System uses a Singly Linked List to implement a Queue.

The following data structures are used in the program:

4.1 Ticket Structure

The ticket structure stores the information related to a customer support request.

```
struct ticket
```

```
{
    int id;
    string name;
    string issue;
```

```
};
```

id: Unique identification number of the ticket.

name: Name of the customer.

issue: Description of the customer's problem.

4.2 Node Structure

The node structure is used to create the linked list.

```
struct node
```

```
{
    ticket data;
    node *next;
};
```

Each node contains:

A ticket object named data.

A pointer named next that stores the address of the next node.

4.3 Front and Rear Pointers

Two global pointers are used to manage the queue:

```
node *front = NULL;
```

```
node *rear = NULL;
```

front points to the first ticket in the queue.

rear points to the last ticket in the queue.

These pointers allow insertion and deletion operations to be performed efficiently while maintaining FIFO order.

Queue Implemented as a Singly Linked List

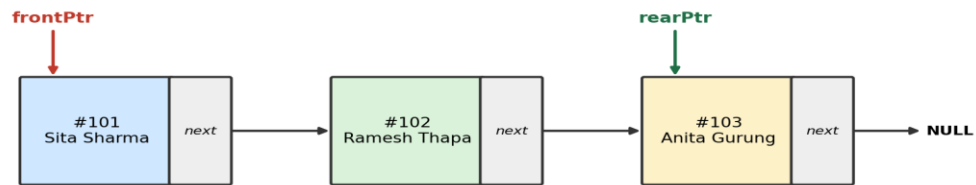


Figure 2. Internal structure of the TicketQueue: each node stores one ticket and a pointer to the next node; the last node's next pointer is NULL.

5. Algorithm

Algorithm: Add Ticket (Enqueue)

1. Create a new node.
2. Store ticket information in the node.
3. Set the next pointer of the node to NULL.
4. If the queue is empty, make both front and rear point to the new node.
5. Otherwise, link the new node after rear and update rear.
6. Display a confirmation message.

Algorithm: Resolve Ticket (Dequeue)

1. Check whether the queue is empty.
2. If empty, display an appropriate message.
3. Otherwise, store the front ticket information.
4. Move front to the next node.
5. Delete the previous front node.
6. If the queue becomes empty, set rear to NULL.
7. Display the resolved ticket information.

Algorithm: View Next Ticket (Peek)

1. Check whether the queue is empty.
2. If empty, display a message.
3. Otherwise, display the ticket at the front of the queue.

Algorithm: Display All Tickets

1. Check whether the queue is empty.
2. If empty, display a message.
3. Otherwise, traverse the linked list from front to rear.
4. Display the details of each ticket.

Algorithm: Count Pending Tickets

1. Traverse the queue or use the maintained count variable.
2. Display the total number of pending tickets.

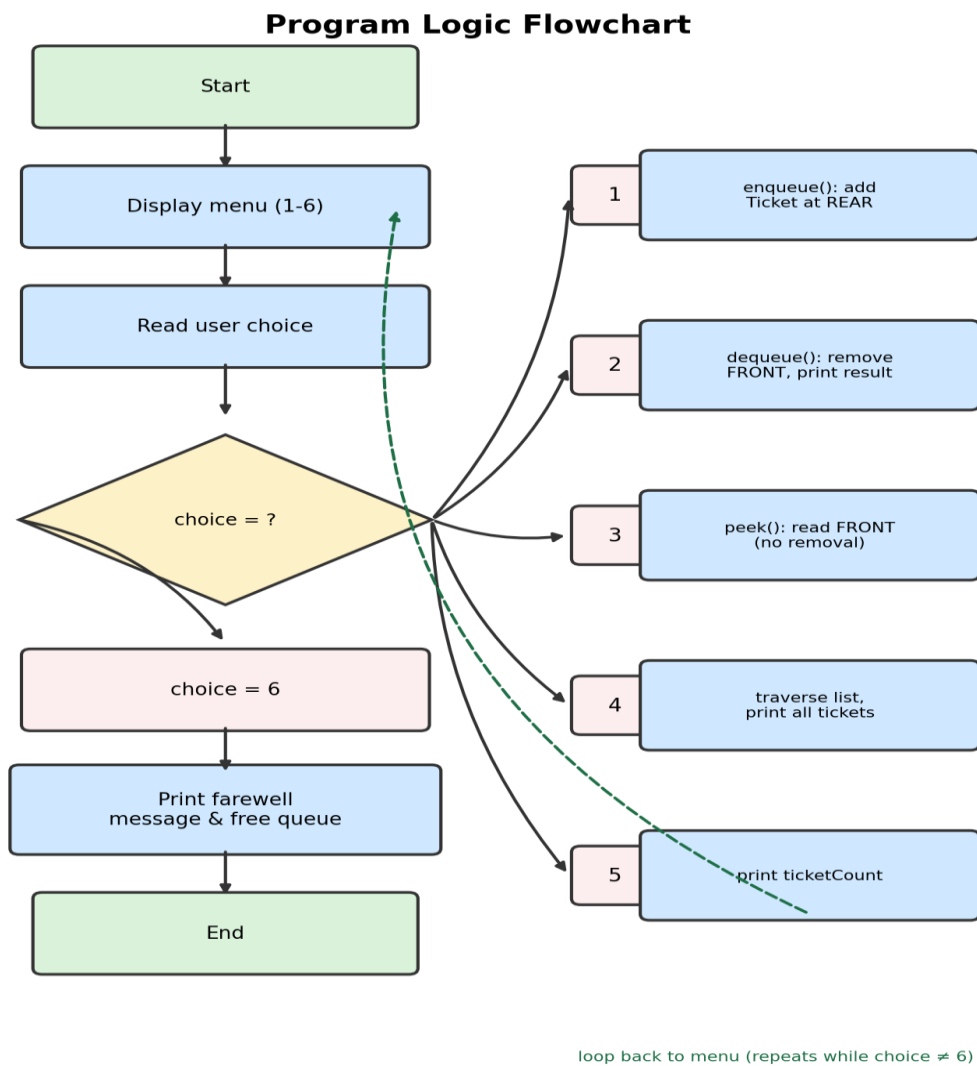


Figure 3. Flowchart of the menu-driven control loop, showing how each user choice maps to a queue operation before looping back to the menu.

6. Complete Source Code

```
#include<iostream>
#include<string>
using namespace std;
struct ticket
{
    int id;
    string name;
    string issue;
};

struct node
{
    ticket data;
    node *next;
};

node *front = NULL;
node *rear = NULL;

void enqueue(ticket T1)
{
    node *N = new node;
    N->data = T1;
    N->next = NULL;

    if(front == NULL)
    {
        front = rear = N;
    }
    else
    {
        rear->next = N;
        rear = N;
    }
}

void dequeue()
{
    if(front == NULL)
    {
        cout << "No Pending Tickets." << endl;
    }
    else
    {
        node *temp = front;

        cout << "Ticket ID : " << temp->data.id << " Resolved." << endl;

        front = front->next;

        if(front == NULL)
        {
            rear = NULL;
        }

        delete temp;
    }
}

void view()
{
    if(front == NULL)
    {
        cout << "No Pending Tickets." << endl;
    }
}
```



```

    }
    else
    {
        cout << "Next Ticket" << endl;
        cout << "Ticket ID: " << front->data.id << endl;
        cout << "Ticket Holder Name: " << front->data.name << endl;
        cout << "Ticket Issue: " << front->data.issue << endl;
    }
}

void display()
{
    if(front == NULL)
    {
        cout << "No Pending Tickets." << endl;
    }
    else
    {
        node *temp = front;

        cout << "\nPending Tickets:\n";

        while(temp != NULL)
        {
            cout << "-----" << endl;
            cout << "Ticket ID: " << temp->data.id << endl;
            cout << "Customer Name: " << temp->data.name << endl;
            cout << "Issue: " << temp->data.issue << endl;

            temp = temp->next;
        }
    }
}

int count()
{
    int c = 0;

    node *temp = front;

    while(temp != NULL)
    {
        c++;
        temp = temp->next;
    }

    return c;
}

int main()
{
    int choice;

    do
    {
        cout << "\n===== Customer Support Ticket Router =====" << endl;
        cout << "1. Add Ticket" << endl;
        cout << "2. Process Ticket" << endl;
        cout << "3. View Next Ticket" << endl;
        cout << "4. Display All Pending Tickets" << endl;
        cout << "5. Display Ticket Count" << endl;
        cout << "6. Exit" << endl;
        cout << "===== " << endl;
        cout << "Enter your choice (1-6): ";

        cin >> choice;
    }
}

```

```

    cout << endl;

    switch(choice)
    {
    case 1:
    {
        ticket T1;

        cout << "Enter the ticket details" << endl;
        cout << "Enter ID: ";
        cin >> T1.id;

        cout << "Enter Customer Name: ";
        cin.ignore();
        getline(cin, T1.name);

        cout << "Enter Issue: ";
        getline(cin, T1.issue);

        cout << endl;

        enqueue(T1);

        cout << "Ticket Added Successfully." << endl;
        break;
    }

    case 2:
    {
        dequeue();
        break;
    }

    case 3:
    {
        view();
        break;
    }

    case 4:
    {
        display();
        break;
    }

    case 5:
    {
        cout << "Total Number of Tickets Pending: " << count() << endl;
        break;
    }

    case 6:
    {
        cout << "Program Terminated." << endl;
        break;
    }

    default:
    {
        cout << "Invalid Choice." << endl;
    }
    }

    } while(choice != 6);

    return 0;
}

```

7. Sample Output

```

===== Customer Support Ticket Router =====
1. Add Ticket
2. Process Ticket
3. View Next Ticket
4. Display All Pending Tickets
5. Display Ticket Count
6. Exit
=====
Enter your choice (1-6): 1

Enter the ticket details
Enter ID: 101
Enter Customer Name: Ram Shrestha
Enter Issue: Internet not working

Ticket Added Successfully.

===== Customer Support Ticket Router =====
Enter your choice (1-6): 1

Enter the ticket details
Enter ID: 102
Enter Customer Name: Sita Gurung
Enter Issue: Slow speed

Ticket Added Successfully.

===== Customer Support Ticket Router =====
Enter your choice (1-6): 3

Next Ticket
Ticket ID: 101
Ticket Holder Name: Ram Shrestha
Ticket Issue: Internet not working

===== Customer Support Ticket Router =====
Enter your choice (1-6): 2

Ticket ID : 101 Resolved.

===== Customer Support Ticket Router =====
Enter your choice (1-6): 4

Pending Tickets:
-----
Ticket ID: 102
Customer Name: Sita Gurung
Issue: Slow speed

===== Customer Support Ticket Router =====
Enter your choice (1-6): 5

Total Number of Tickets Pending: 1

===== Customer Support Ticket Router =====
Enter your choice (1-6): 6

Program Terminated.

```

8. Conclusion

The Customer Support Ticket Router system successfully demonstrates the practical implementation of a queue using a singly linked list. It follows the FIFO (First In First Out) principle, ensuring that tickets are processed in the order they are received. The system efficiently handles dynamic memory allocation, allowing insertion (enqueue), deletion (dequeue), viewing, and counting of tickets without a fixed size limitation. This project helps in understanding how linked lists can be used to build real-world applications such as task scheduling and customer support management systems.

Additionally, this project improves understanding of pointer manipulation and dynamic memory management in C++. It also strengthens problem-solving skills by applying data structure concepts to a real-world scenario. Overall, it provides a clear foundation for developing more advanced systems such as priority-based ticket handling or multi-agent support systems.